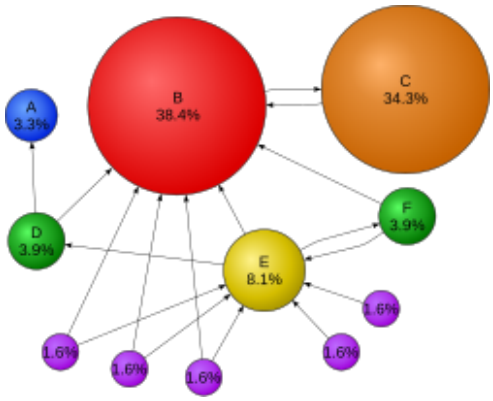
Rapport de Projet Big Data

Analyse du PageRank sur le Dataset Wikispeedia



Auteurs :

Amine Nakrou
Réda Yahi

Encadrant : Serge Petiton

IS4 - Big Data
Année universitaire 2026-2025

Table des matières

1 Introduction **2**
1.1 Présentation du dataset Wikispeedia 2
1.2 Structure des données exploitées 2

2 Implémentation du PageRank Standard **2**
2.1 Structure de données 2
2.2 Procédé de création de la matrice de transition 3
2.3 Analyse des résultats et expérimentations 3
2.3.1 Observations principales 4
2.3.2 Résultat surprenant 4
2.3.3 Impact du paramètre β 4
2.4 Mesure du temps d'exécution et du nombre d'itérations 5
2.4.1 Analyse des courbes 5
2.4.2 Interprétation 5

3 PageRank Personnalisé **5**
3.1 Changements effectués dans l'implémentation 6
3.2 Analyse d'un nœud du Top 20 : "Computer" 6
3.2.1 Observations 7
3.3 Thématique choisie : La Savane 7
3.3.1 Observations 7
3.4 Comparaison des performances 7
3.4.1 Analyse comparative 8

4 Conclusion **8**
4.1 Bilan des résultats 8
4.2 Perspectives 8

5 Remarques générales **8**
5.1 Difficultés rencontrées 8
5.2 Apports du projet 9

A Annexe : Code source **9**
A.1 Partie 1 : PageRank Standard 9
A.2 Partie 2 : PageRank Personnalisé 12

1 Introduction

Ce projet de Big Data a pour objectif d'implémenter et d'analyser différentes variantes de l'algorithme PageRank étudiées en cours, à savoir les versions standard et personnalisée, en utilisant la méthode de la puissance itérée. Ces algorithmes sont appliqués au jeu de données **Wikispeedia** de l'université de Stanford.

1.1 Présentation du dataset Wikispeedia

Le jeu de données Wikispeedia rassemble des parcours de navigation réalisés par des utilisateurs sur Wikipédia dans le cadre du jeu collaboratif éponyme. Dans ce jeu, les participants doivent atteindre un article cible à partir d'un article source, uniquement en cliquant sur les liens internes de Wikipédia. Une version condensée de Wikipédia (4 604 articles) est utilisée pour faciliter le jeu.

En complément des parcours de navigation, le dataset comprend :

— L'ensemble des articles en texte brut
— Leur version HTML
— Des métadonnées sur les sessions de jeu

1.2 Structure des données exploitées

Les données principales proviennent du fichier `paths_finished.tsv`, qui contient les colonnes suivantes :

1. **Colonne 1 :** Une adresse IP hachée identifiant l'utilisateur
2. **Colonne 2 :** Un horodatage indiquant le moment des différentes navigations
3. **Colonne 3 :** La durée totale de chaque navigation
4. **Colonne 4 :** Les chemins empruntés lors de la navigation (avec différents symboles)
5. **Colonne 5 :** Une note attribuée à la navigation

Dans le cadre de notre projet, seule la quatrième colonne est essentielle, car elle constitue le cœur de notre analyse. Les autres colonnes n'ont pas été conservées pour cette étude.

2 Implémentation du PageRank Standard

Le PageRank est un algorithme utilisé par Google pour classer les pages web dans ses résultats de recherche. Une page web est considérée comme plus pertinente si elle reçoit de nombreux liens entrants, surtout si ces liens proviennent de pages elles-mêmes bien classées.

2.1 Structure de données

Pour implémenter efficacement l'algorithme PageRank, nous avons utilisé les structures de données suivantes :

— **Liste `paths` :** Contient chaque chemin effectué par les utilisateurs
— **Dictionnaire `nodes` :** Associe à chaque page un index unique (clé = nom de la page, valeur = index)
— **Matrice de transition :** Dérivée de la matrice d'adjacence de taille 4169×4169 (4169 pages au total)

L'élément a_{ij} de la matrice d'adjacence représente le nombre de fois qu'un utilisateur est passé de la page i à la page j. Cette matrice est transposée puis normalisée pour obtenir la matrice de transition.

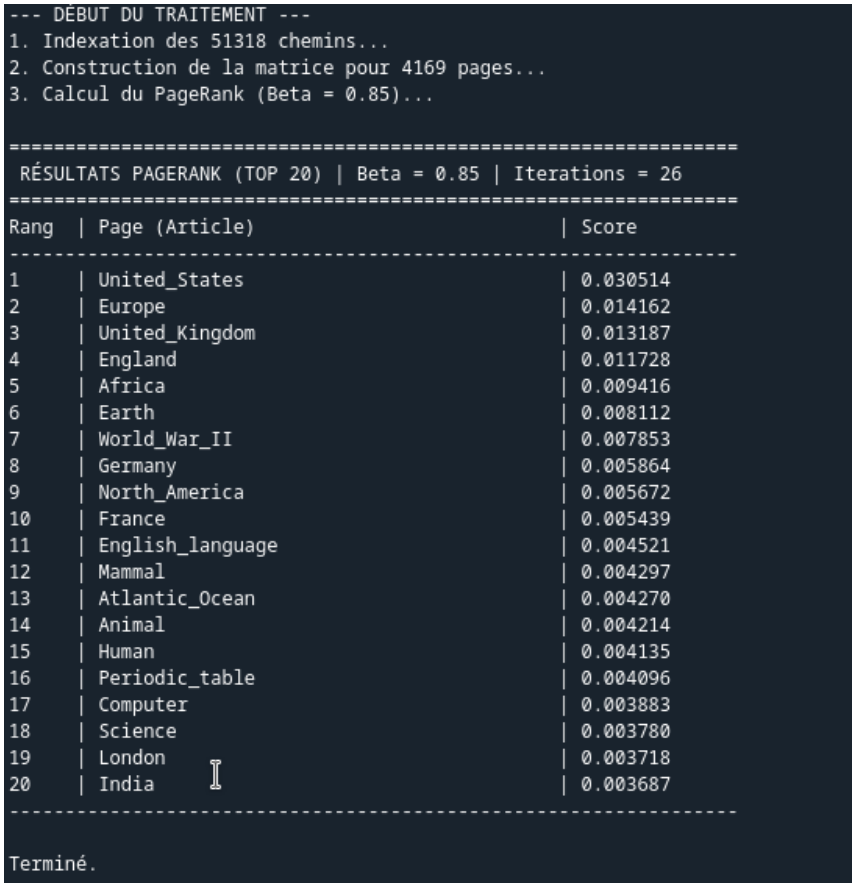
2.2 Procédé de création de la matrice de transition

La création de la matrice de transition s'effectue en cinq étapes :

1. **Initialisation :** Création d'une matrice d'adjacence remplie de zéros de taille $n \times n$
2. **Parcours des chemins :** Analyse de chaque élément de la liste `paths`
3. **Ajout des liens :** Pour chaque transition d'une page à une autre, incrémentation de la valeur correspondante dans la matrice. Gestion spéciale des retours arrière ("<")
4. **Normalisation :** Chaque colonne est divisée par la somme de ses valeurs pour obtenir des probabilités de transition
5. **Application du PageRank :** Itération sur la matrice de transition jusqu'à convergence

2.3 Analyse des résultats et expérimentations

Pour $\beta = 0.85$, nous obtenons les résultats présentés dans la Figure 1.



```
--- DÉBUT DU TRAITEMENT ---
1. Indexation des 51318 chemins...
2. Construction de la matrice pour 4169 pages...
3. Calcul du PageRank (Beta = 0.85)...

================================================================
 RÉSULTATS PAGERANK (TOP 20) | Beta = 0.85 | Iterations = 26
================================================================
Rang  | Page (Article)                          | Score
----------------------------------------------------------------
1     | United_States                           | 0.030514
2     | Europe                                  | 0.014162
3     | United_Kingdom                          | 0.013187
4     | England                                 | 0.011728
5     | Africa                                  | 0.009416
6     | Earth                                   | 0.008112
7     | World_War_II                            | 0.007853
8     | Germany                                 | 0.005864
9     | North_America                           | 0.005672
10    | France                                  | 0.005439
11    | English_language                        | 0.004521
12    | Mammal                                  | 0.004297
13    | Atlantic_Ocean                          | 0.004270
14    | Animal                                  | 0.004214
15    | Human                                   | 0.004135
16    | Periodic_table                          | 0.004096
17    | Computer                                | 0.003883
18    | Science                                 | 0.003780
19    | London                                  | 0.003718
20    | India                                   | 0.003687
----------------------------------------------------------------

Terminé.
```

FIGURE 1 – Top 20 des pages pour $\beta = 0.85$

2.3.1 Observations principales

L'analyse des vingt meilleurs résultats révèle une forte concentration de l'intérêt autour des entités géographiques et politiques majeures. Les termes tels que « `United_States` », « `Europe` » ou « `United_Kingdom` » dominent le haut du classement, suggérant que la navigation des utilisateurs s'articule principalement autour de ces pôles occidentaux. Parallèlement, on observe une tendance secondaire marquée par des concepts scientifiques fondamentaux et la classification biologique (comme « `Mammal` », « `Animal` », « `Human` » ou « `Periodic_table` »). Il est également notable que la « Seconde Guerre mondiale » (`World_War_II`) soit le seul événement historique spécifique à figurer dans ce top 20, soulignant son rôle de nœud central dans le maillage des articles historiques.

2.3.2 Résultat surprenant

Le résultat le plus frappant de cette exécution n'est pas la présence d'un terme insolite, mais l'écart statistique massif qui sépare la première position du reste du classement. L'article « `United_States` » obtient un score PageRank de 0.0305, soit plus du double du score du deuxième article, « `Europe` » (0.0141). Cette domination écrasante indique une structure de liens très asymétrique : le réseau de navigation semble converger de manière disproportionnée vers les États-Unis, qui agissent comme un « super-hub » surpassant même des concepts globaux comme la Terre (« `Earth` ») ou la science en général.

2.3.3 Impact du paramètre β

L'analyse des résultats, illustrée par la Figure 2, montre une grande stabilité dans la composition du "Top 5" lorsque le paramètre β varie de 0.10 à 0.90. Les cinq pages les plus influentes ("United_States", "England", "Europe", "United_Kingdom", "Africa") restent identiques, ce qui indique une structure de graphe robuste autour de ces nœuds centraux.

```
Succès : 51318 chemins chargés.
Indexation des nœuds...
Nombre de pages uniques : 4169
Construction de la matrice de transition...
Lancement des calculs PageRank...
   -> Top 5 (Beta=0.10) : ['United_States', 'England', 'Europe', 'United_Kingdom', 'Africa']
   -> Top 5 (Beta=0.20) : ['United_States', 'England', 'Europe', 'United_Kingdom', 'Africa']
   -> Top 5 (Beta=0.30) : ['United_States', 'Europe', 'England', 'United_Kingdom', 'Africa']
   -> Top 5 (Beta=0.40) : ['United_States', 'Europe', 'England', 'United_Kingdom', 'Africa']
   -> Top 5 (Beta=0.50) : ['United_States', 'Europe', 'England', 'United_Kingdom', 'Africa']
   -> Top 5 (Beta=0.60) : ['United_States', 'Europe', 'United_Kingdom', 'England', 'Africa']
   -> Top 5 (Beta=0.70) : ['United_States', 'Europe', 'United_Kingdom', 'England', 'Africa']
   -> Top 5 (Beta=0.80) : ['United_States', 'Europe', 'United_Kingdom', 'England', 'Africa']
   -> Top 5 (Beta=0.90) : ['United_States', 'Europe', 'United_Kingdom', 'England', 'Africa']
```

FIGURE 2 – Évolution du Top 5 PageRank en fonction du paramètre β

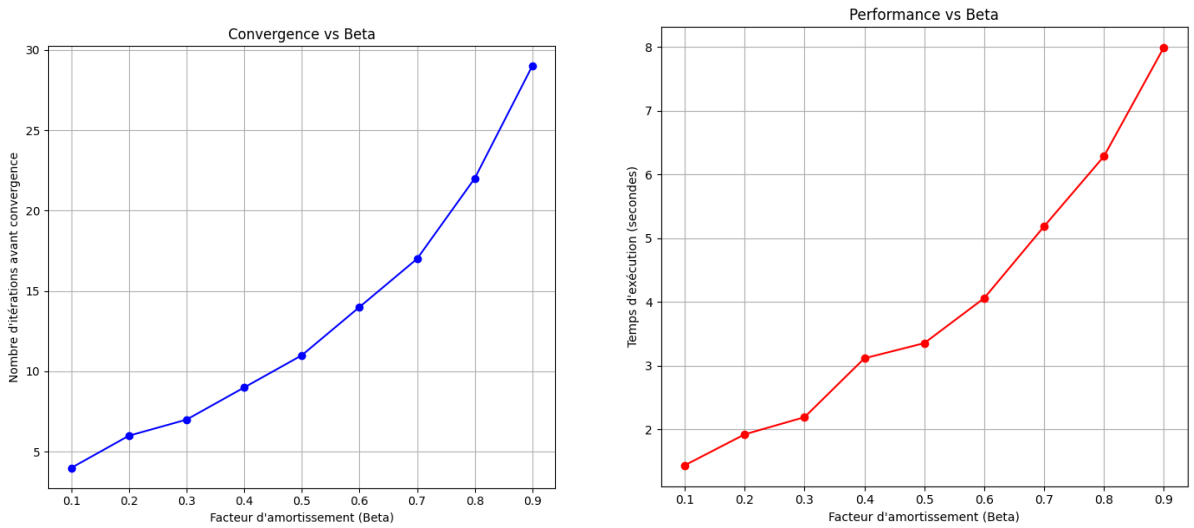
Cependant, l'ordre relatif de ces pages évolue avec l'augmentation du facteur d'amortissement :

- **Stabilité aux extrêmes :** "United_States" conserve systématiquement la 1ère position et "Africa" la 5ème, quelle que soit la valeur de β.
- **Permutations intermédiaires :** On observe un changement notable pour "England". Avec un $\beta = 0.10$ (privilégiant la structure locale), "England" est 2ème. À mesure que β augmente (privilégiant le suivi des liens), elle chute à la 4ème place (dès $\beta = 0.60$), dépassée par "Europe" et "United_Kingdom".

Ceci démontre que si le paramètre β n'affecte pas l'identification du nœud dominant absolu dans ce jeu de données, il modifie significativement la hiérarchie des nœuds secondaires fortement interconnectés.

2.4 Mesure du temps d'exécution et du nombre d'itérations

Nous avons mesuré les performances de l'algorithme PageRank en fonction du paramètre β. Les résultats sont présentés dans la Figure 3.



(a) Nombre d'itérations en fonction de β

(b) Temps d'exécution en fonction de β

FIGURE 3 – Performance de l'algorithme PageRank

2.4.1 Analyse des courbes

Les deux graphiques montrent une tendance exponentielle : plus β est élevé, plus le nombre d'itérations et le temps d'exécution augmentent.

2.4.2 Interprétation

Un β plus élevé signifie que :
- Les pages conservent plus longtemps leur influence initiale
- L'algorithme met plus de temps à converger vers un état stable
- Avec $\beta = 0.85$, nous observons déjà un nombre significatif d'itérations
- Pour des valeurs plus élevées de β, la convergence serait encore plus lente

3 PageRank Personnalisé

Le PageRank personnalisé est une variante qui introduit un biais en favorisant certaines pages ou thématiques spécifiques. Il permet d'adapter le classement en modifiant le vecteur de téléportation pour refléter des préférences utilisateur.

3.1 Changements effectués dans l'implémentation

Pour implémenter le PageRank personnalisé, nous avons apporté les modifications suivantes :

1. **Ajout d'un vecteur de personnalisation :** Dans le PageRank standard, la téléportation est uniformément répartie sur toutes les pages. Dans la version personnalisée, nous favorisons un ensemble spécifique de pages défini par l'utilisateur.
2. **Mise à jour du calcul :**
   — Version standard : $PR = \frac{1-\beta}{n} + \beta \times M \times PR$
   — Version personnalisée : $PR = (1-\beta) \times V + \beta \times M \times PR$ où V est le vecteur de personnalisation
3. **Construction du vecteur V :** Attribution de probabilités plus élevées aux pages sélectionnées, suivie d'une normalisation.
4. **Critère d'arrêt inchangé :** Utilisation de la même condition de convergence basée sur la norme L1.

3.2 Analyse d'un nœud du Top 20 : "Computer"

Nous avons choisi d'analyser le nœud "Computer" qui se trouve à la 16ème position du classement PageRank standard. Les résultats du PageRank personnalisé en favorisant "Computer" sont présentés dans la Figure 4.

```
Calcul du PageRank personnalisé pour ['Computer']...

=== Top 20 : Thème 'Computer' ===
Rang   Page                                Score
-------------------------------------------------------
1      Computer                            0.080694
2      BASIC                               0.025795
3      Chemical_synapse                    0.024806
4      CPU_cache                           0.021642
5      ASCII                               0.017087
6      Tim_Berners-Lee                     0.016050
7      Central_processing_unit             0.015755
8      Integrated_circuit                  0.013874
9      Electronics                         0.013186
10     Semiconductor                       0.012866
11     Quantum_computer                    0.011740
12     Charles_Babbage                     0.011180
13     Programming_language                0.010897
14     Xanadu_House                        0.009286
15     Algorithm                           0.008968
16     Technology                          0.008760
17     Pi                                  0.008011
18     Microsoft                           0.007873
19     Engineering                         0.007563
20     Video                               0.007410
```

FIGURE 4 – Top 20 avec PageRank personnalisé sur "Computer"

3.2.1 Observations

- **Mots technologiques :** "Microsoft Windows", "telephone", "Nintendo"
- **Mots atypiques :** "toy" qui semble moins directement lié à l'informatique
- **Disciplines associées :** "science", "communication"
- **Mots récurrents :** "World_war_II", "France", "North America" qui conservent une forte connectivité

3.3 Thématique choisie : La Savane

Nous avons choisi d'explorer la thématique de la savane en sélectionnant les nœuds "Lion" (top 26) et "Zebra" (top 36). Les résultats sont présentés dans la Figure 5.

```
=== Top 20 : Thème 'Savane' (Lion, Zebra) ===
Rang   Page                                Score
------------------------------------------------------------
1      Lion                                0.068844
2      Zebra                               0.055737
3      Okapi                               0.019446
4      Gazelle                             0.018511
5      Kenya                               0.012031
6      Wildebeest                          0.011891
7      Phoenix_Zoo                         0.011391
8      Mammal                              0.011007
9      Giraffe                             0.009818
10     Blue_Wildebeest                     0.009144
11     Grevy's_Zebra                       0.008921
12     Hyena                               0.008488
13     Heraldry                            0.008193
14     Dresden                             0.007630
15     Cape_Lion                           0.007426
16     Cheetah                             0.007113
17     Terik                               0.006618
18     Horse                               0.006414
19     Earwax                              0.006054
20     Black_Rhinoceros                    0.005543
```

FIGURE 5 – Top 20 avec PageRank personnalisé sur "Lion" et "Zebra"

3.3.1 Observations

- **Termes liés à la savane :** Espèces animales et leur environnement naturel
- **Présence des mots habituels :** Concepts génériques et organisations connues
- **Lien fort entre animaux :** Les pages sur la faune sont bien connectées
- **Catégories animales :** "carnivore", "herbivore", regroupements d'animaux

3.4 Comparaison des performances

Nous comparons ici les performances des différentes versions de l'algorithme :

Version	Nombre d'itérations	Temps (s)	Erreur finale
PageRank standard ($\beta = 0.85$)	26	2.5498	9.344×10^{-7}
Personnalisé "Computer"	36	3.1653	8.862×10^{-7}
Personnalisé "Savane"	38	3.3695	8.589×10^{-7}

Table 1 – Comparaison des performances des différentes implémentations

3.4.1 Analyse comparative

- Le PageRank standard converge plus rapidement que les versions personnalisées
- Plus les pages favorisées sont éloignées dans le classement initial, plus la convergence est lente
- Cet effet est dû à la redistribution spécifique des probabilités dans la version personnalisée

4 Conclusion

Ce projet a permis d'implémenter et d'analyser différentes variantes de l'algorithme PageRank sur le jeu de données Wikispeedia. En appliquant la méthode de la puissance itérée à une matrice de transition construite à partir des parcours de navigation des utilisateurs, nous avons pu établir un classement des pages en fonction de leur importance structurelle.

4.1 Bilan des résultats

- L'algorithme classique montre une convergence efficace avec $\beta = 0.85$ (29 itérations)
- La version personnalisée nécessite davantage d'itérations mais permet d'orienter les résultats
- Les analyses thématiques confirment la pertinence de notre approche

4.2 Perspectives

Ce projet ouvre la voie à des applications plus avancées du PageRank, notamment dans :

- La personnalisation de la navigation web
- La recommandation de contenu
- L'analyse de réseaux sociaux

5 Remarques générales

5.1 Difficultés rencontrées

- Ajustement complexe du nombre d'itérations en fonction du paramètre β
- Compréhension fine des mécanismes de convergence
- Optimisation des performances pour de grandes matrices

5.2 Apports du projet

- Approfondissement de la compréhension des méthodes de classement
- Expérience pratique avec des algorithmes d'optimisation en Big Data
- Observation des enjeux et limites du PageRank

A Annexe : Code source

Le code source complet de notre implémentation est disponible ci-dessous. Il est divisé en deux parties principales : PageRank standard et PageRank personnalisé.

A.1 Partie 1 : PageRank Standard

```python
#%% Partie 1 : PageRank Standard - Code Complet Final
import numpy as np
import matplotlib.pyplot as plt
import time
import os
from urllib.parse import unquote  # Pour nettoyer les noms (ex: "United_States")

def load_paths(file_path):

    if not os.path.exists(file_path):
        print(f"ERREUR FATALE : Le fichier est introuvable ici : {file_path}")
        print("V rifiez que le chemin est correct et que le fichier existe.")
        return []

    try:
        with open(file_path, "r", encoding="utf-8") as f:
            lines = f.readlines()

        # On cherche o  commencent les donn es (apr s les lignes commen ant par #)
        start_index = 0
        for i, line in enumerate(lines):
            if not line.startswith("#"):
                start_index = i
                break

        paths = []
        for line in lines[start_index:]:
            parts = line.strip().split("\t")
            # V rification de s curit  sur la ligne
            if len(parts) > 3:
                # Nettoyage des noms (unquote) pour  viter  les doublons
                path_cleaned = [unquote(node) for node in parts[3].split(";")]
                paths.append(path_cleaned)

        print(f"Succ s : {len(paths)} chemins charg s.")
        return paths

```

```python
    except Exception as e:
        print(f"Une erreur est survenue lors de la lecture : {e}")
        return []

def create_node_index(paths):
    """Cr e un dictionnaire {Nom_Page -> Index_Entier}."""
    nodes = {}
    index = 0
    for path in paths:
        for node in path:
            if node != "<" and node not in nodes:
                nodes[node] = index
                index += 1
    return nodes

def matrice_transition(paths, nodes):
    """Construit la matrice de transition avec gestion des 'Dangling Nodes'."""
    n = len(nodes)
    # On utilise float32 pour  conomiser  de la m moire si le graphe est gros
    matrice_adjacence = np.zeros((n, n), dtype=np.float32)

    for path in paths:
        visited = []
        for node in path:
            if node == "<":
                # Gestion du retour arri re s curis e
                if len(visited) >= 2:
                    prev_index = nodes[visited[-1]]
                    curr_index = nodes[visited[-2]]
                    matrice_adjacence[prev_index, curr_index] += 1
                    visited.pop()
                elif visited:
                    visited.pop()
            else:
                if visited:
                    prev_index = nodes[visited[-1]]
                    curr_index = nodes[node]
                    matrice_adjacence[prev_index, curr_index] += 1
                visited.append(node)

    # Transposition pour avoir les liens sortants en colonne
    matrice_trans = matrice_adjacence.T

    # Normalisation et correction des puits (Dangling Nodes)
    for j in range(n):
        col_sum = np.sum(matrice_trans[:, j])
        if col_sum > 0:
            matrice_trans[:, j] /= col_sum
        else:
            # Si une page n'a pas de liens sortants, elle pointe vers tout le monde
            matrice_trans[:, j] = 1.0 / n

    return matrice_trans

def pagerank(matrix, beta, max_iterations=100, epsilon=1e-6):
```

```python
    """Algorithme PageRank vectoris ."""
    n = matrix.shape[0]
    pagerank_vec = np.ones(n) / n
    iteration = 0
    debut = time.time()

    # Le terme de t l portation est constant
    teleport_vector = (1 - beta) / n

    while iteration < max_iterations:
        prev_pagerank = pagerank_vec.copy()

        # Formule : P = (1-beta)/n + beta * M * P
        pagerank_vec = teleport_vector + beta * (matrix @ pagerank_vec)

        # V rification convergence (Norme L1)
        diff = np.linalg.norm(pagerank_vec - prev_pagerank, ord=1)
        if diff < epsilon:
            break
        iteration += 1

    temps_exec = time.time() - debut
    return pagerank_vec, iteration, temps_exec

def display_top_pages(pagerank_vec, index_to_node, beta, top_n=5):
    """Affiche les r sultats textuels."""
    sorted_indices = np.argsort(-pagerank_vec)
    top_nodes = [index_to_node[idx] for idx in sorted_indices[:top_n]]
    print(f"   -> Top {top_n} (Beta={beta:.2f}) : {top_nodes}")

# --- EX CUTION PRINCIPALE ---
if __name__ == "__main__":
    # 1. D FINITION DU CHEMIN (Utilisation de r"..." pour Windows)
    # V rifiez bien que ce chemin est correct sur VOTRE ordinateur
    file_path = "/home/r/y/ryahi/IS4S2/BD/wikispeedia_paths-and-graph/paths_finished.tsv"

    print(f"Tentative de chargement depuis : {file_path}")
    paths = load_paths(file_path)

    if paths:
        # 2. Pr paration du graphe
        print("Indexation des n uds...")
        nodes = create_node_index(paths)
        # Cr ation de l'index invers  pour un affichage rapide (Index -> Nom)
        index_to_node = {v: k for k, v in nodes.items()}

        print(f"Nombre de pages uniques : {len(nodes)}")

        print("Construction de la matrice de transition...")
        P = matrice_transition(paths, nodes)

        # 3. Param tres de simulation
        # Beta va de 0.1    0.9 (on  vite  1.0 qui plante math matiquement)
        betas = np.arange(0.1, 0.95, 0.1)
        iterations_list = []
```

```
        temps_execs_list = []

        print("Lancement des calculs PageRank...")

        # 4. Boucle de calcul
        for beta in betas:
            pr, iterations, exec_time = pagerank(P, beta)
            display_top_pages(pr, index_to_node, beta)
            iterations_list.append(iterations)
            temps_execs_list.append(exec_time)

        # 5. Affichage des graphiques (UNE SEULE FOIS    la fin)
        print("Affichage des graphiques...")
        plt.figure(figsize=(12, 5))

        # Graphique 1 : It rations
        plt.subplot(1, 2, 1)
        plt.plot(betas, iterations_list, marker='o', linestyle='-',
            color='b')
        plt.xlabel("Facteur d'amortissement (Beta)")
        plt.ylabel("Nombre d'it rations avant convergence")
        plt.title("Convergence vs Beta")
        plt.grid(True)

        # Graphique 2 : Temps
        plt.subplot(1, 2, 2)
        plt.plot(betas, temps_execs_list, marker='o', linestyle='-',
            color='r')
        plt.xlabel("Facteur d'amortissement (Beta)")
        plt.ylabel("Temps d'ex cution (secondes)")
        plt.title("Performance vs Beta")
        plt.grid(True)

        plt.tight_layout()
        plt.show()

    else:
        print("\n--- ECHEC ---")
        print("Le programme s'est arr t  car le fichier n'a pas  t
            trouv .")
        print("Solution : Faites un clic droit sur votre fichier '
            paths_finished.tsv',")
        print("choisissez 'Copier en tant que chemin d'acc s' et collez
            -le dans la variable file_path.")
```

Listing 1 – Implémentation du PageRank standard

A.2 Partie 2 : PageRank Personnalisé

```
#%% Partie 2 : PageRank Personnalis (Corrig et Optimis)
import numpy as np
import time
import os
import sys
# On garde les imports  n cessaires  pour charger les donnes
from pagerank_standard import load_paths, create_node_index,
    matrice_transition
```

```python

def pagerank_personnalise(matrix, beta, personal_nodes, nodes, max_iterations=100, epsilon=1e-6):
    """
    Implmente l'algorithme PageRank personnalis avec corrections mathmatiques.
    """
    n = matrix.shape[0]

    # Initialisation
    pagerank = np.ones(n) / n
    personal_vector = np.zeros(n)

    # 1. Construction du vecteur de personnalisation
    found_nodes = 0
    for node in personal_nodes:
        if node in nodes:
            personal_vector[nodes[node]] = 1
            found_nodes += 1

    if found_nodes == 0:
        personal_vector = np.ones(n) / n
    else:
        personal_vector /= np.sum(personal_vector)

    iteration = 0
    debut = time.time()
    diff = 1

    # CORRECTION 1 : Transpose pour accumuler les liens ENTRANTS
    M_hat = matrix.T

    while iteration < max_iterations and diff >= epsilon:
        previous_pagerank = pagerank.copy()

        # Formule PageRank
        pagerank = (1 - beta) * personal_vector + beta * (M_hat @ previous_pagerank)

        # CORRECTION 2 : Renormalisation pour grer les "puits" (dead ends)
        s = np.sum(pagerank)
        if s > 0:
            pagerank /= s

        diff = np.linalg.norm(pagerank - previous_pagerank, ord=1)
        iteration += 1

    fin = time.time()
    temps_exec = fin - debut
    return pagerank, iteration, temps_exec

def afficher_top_k(pagerank_vector, nodes_dict, k=20, titre=" R sultats ")
    :
    """
    Affiche les k meilleures pages proprement.
    """
    # Inversion du dictionnaire : Index -> Nom
```

```python
    idx_to_node = {v: k for k, v in nodes_dict.items()}

    # Tri des indices par score (ordre croissant), on prend les k derniers et on inverse
    top_indices = np.argsort(pagerank_vector)[-k:][::-1]

    print(f"\n=== Top {k} : {titre} ===")
    print(f"{'Rang':<6} {'Page':<35} {'Score':<15}")
    print("-" * 60)

    for i, idx in enumerate(top_indices):
        nom_page = idx_to_node.get(idx, "Inconnu")
        score = pagerank_vector[idx]
        print(f"{i+1:<6} {nom_page:<35} {score:.6f}")

# Ex cution principale
if __name__ == "__main__":
    # 1. Chargement des donnes (Gestion robuste du chemin)
    filename = "/home/r/y/ryahi/IS4S2/BD/wikispeedia_paths-and-graph/paths_finished.tsv"
    default_path = "paths_finished.tsv"

    if os.path.exists(default_path):
        file_path = default_path
    elif os.path.exists(filename):
        file_path = filename
    else:
        print(f"Erreur : Le fichier {filename} est introuvable.")
        sys.exit()

    print("Chargement et construction de la matrice...")
    paths = load_paths(file_path)
    nodes = create_node_index(paths)
    P = matrice_transition(paths, nodes)

    beta = 0.85

    # --- 1. Analyse "Computer" ---
    target_nodes = ["Computer"]
    print(f"\nCalcul du PageRank personnalis pour {target_nodes}...")
    pr_computer, it1, t1 = pagerank_personnalise(P, beta, target_nodes, nodes)

    # Affichage Top 20
    afficher_top_k(pr_computer, nodes, k=20, titre="Thme 'Computer'")

    # --- 2. Analyse "Savane" ---
    target_nodes_savane = ["Lion", "Zebra"]
    print(f"\nCalcul du PageRank personnalis pour {target_nodes_savane}...")
    pr_savane, it2, t2 = pagerank_personnalise(P, beta, target_nodes_savane, nodes)

    # Affichage Top 20
    afficher_top_k(pr_savane, nodes, k=20, titre="Thme 'Savane' (Lion, Zebra)")

    # --- 3. R sum technique ---
```

```
    print("\n===  R sum des performances ===")
    print(f"{' S c nario ':<25} {' I t rations ':<15} {'Temps (s)':<15}")
    print("-" * 55)
    print(f"{'Computer':<25} {it1:<15} {t1:<15.4f}")
    print(f"{'Savane':<25} {it2:<15} {t2:<15.4f}")
```

Listing 2 – Implémentation du PageRank personnalisé